

Simulation Modeling and Analysis

Dr. Md. Nasim Adnan

The Nature of Simulation

- This is a course about techniques for using computers to imitate, or simulate, the operations of various kinds of real-world facilities or processes (in other words, called systems).
- ***A system is defined to be a collection of entities, e.g., people or machines, that interact together toward the accomplishment of some logical end.***

System

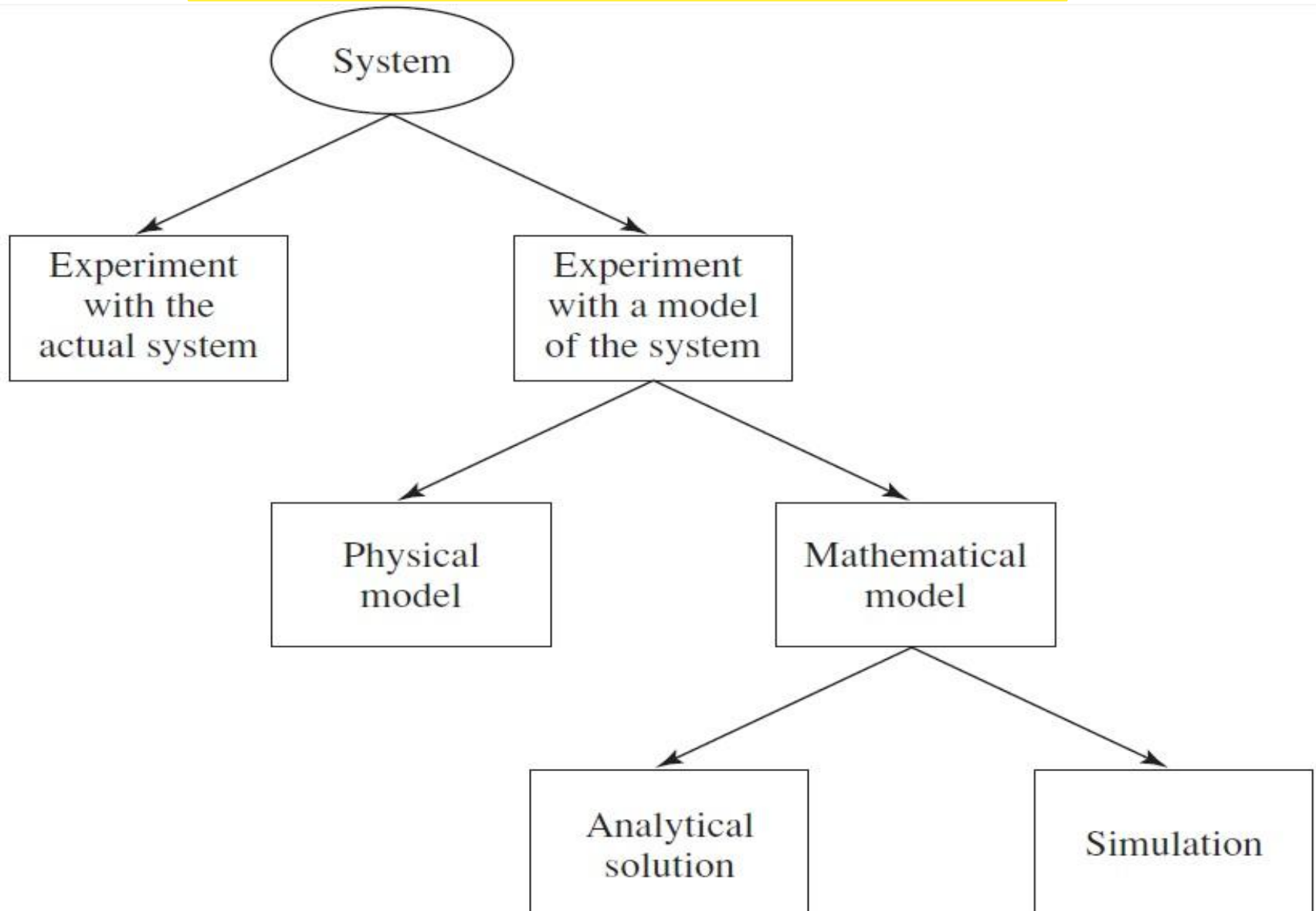
- In practice, what is meant by “the system” depends on the objectives of a particular study. The collection of entities that comprise a system for one study might be only a subset of the overall system for another.
- For example, if one wants to study a bank to determine the number of tellers needed to provide adequate service for customers, the system can be defined to be that portion of the bank consisting of the tellers and the customers waiting in line or being served.
- If, on the other hand, the loan officer and the safe-deposit boxes are to be included, the definition of the system must be expanded in an obvious way.

System

- We define the ***state*** of a system to be that collection of variables necessary to describe a system at a particular time, relative to the objectives of a study.
- In a study of a bank, examples of possible state variables are the number of busy tellers, the number of customers in the bank, and the time of arrival of each customer in the bank.
- We categorize systems to be of two types, discrete and continuous. A discrete system is one for which the state variables change instantaneously at separated points in time.

- A bank is an example of a discrete system, since state variables— e.g., the number of customers in the bank—change only when a customer arrives or when a customer finishes being served and departs.
- A continuous system is one for which the state variables change continuously with respect to time. An airplane moving through the air is an example of a continuous system, since state variables such as position can change continuously with respect to time.
- Few systems in practice are wholly discrete or wholly continuous...

Ways to Study a System



Experiment with the Actual System vs. Experiment with a Model of the System

- If it is possible (and cost-effective) to change the system physically and then let it run under the new conditions, it is usually better to do that. This is because there is no doubt about whether the results we study are valid.
- However, in most cases this is not practical, because such an experiment can be too expensive or can disturb the system too much.

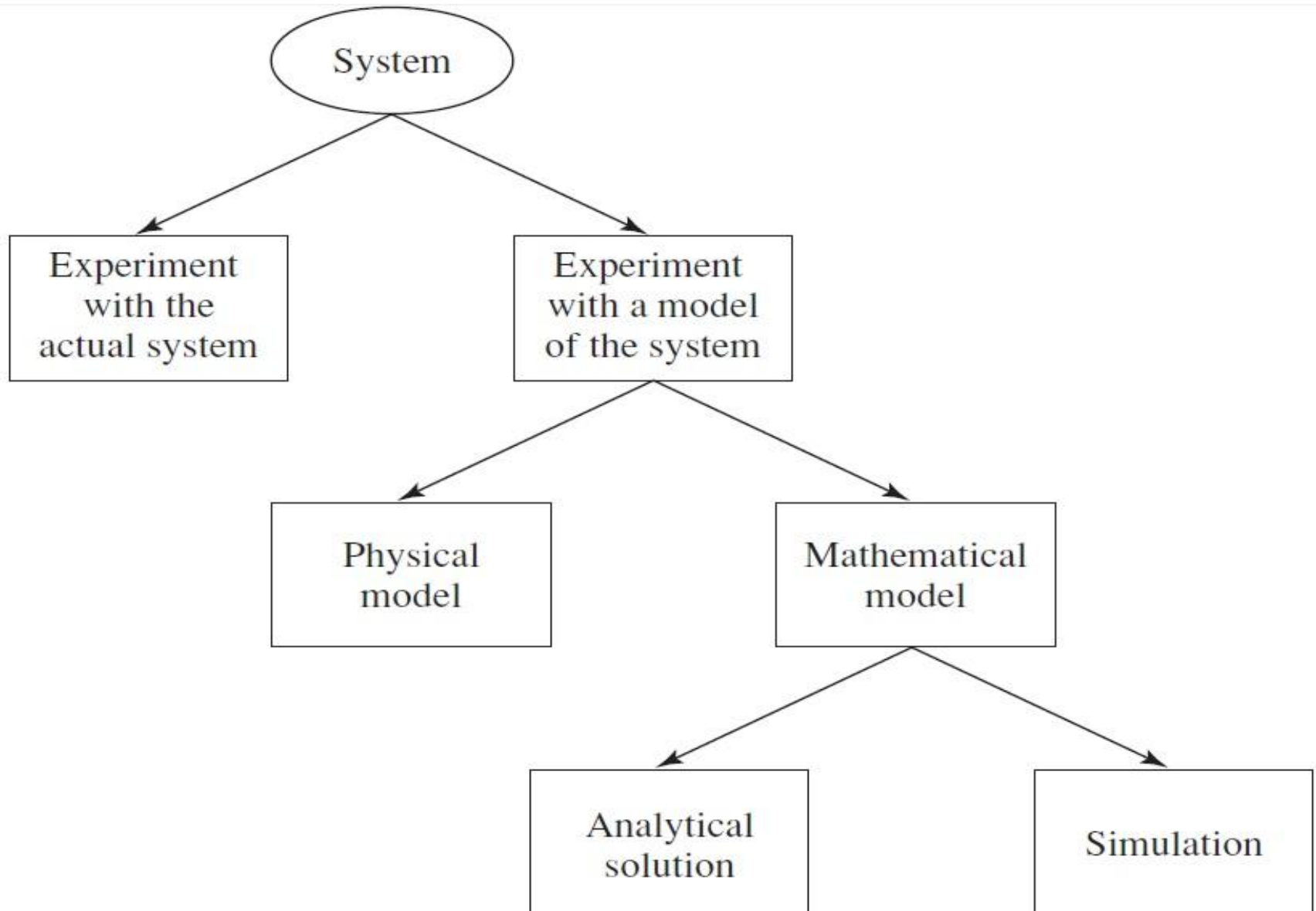
- For example, a bank may think about reducing the number of tellers to lower costs, but actually doing this could cause long waiting times for customers and make them unhappy.
- In some cases, the “system” may not even exist yet, but we still want to study it in different proposed designs to decide how it should be built in the first place.

Physical Model vs. Mathematical Model

- To most people, the word “model” brings to mind things like clay cars tested in wind tunnels, flight simulators used to train pilots, or small ship models moving around in a pool.
- More examples include small tabletop models of material-handling systems, and in at least one case, a full-size physical model of a fast-food restaurant with real (and probably hungry) people inside.

- But the vast majority of models built for such purposes are mathematical, representing a system in terms of logical and quantitative relationships.
- That system are then manipulated and changed to see how the model reacts, which helps us understand how the real system would behave.

Ways to Study a System



Analytical Solution vs. Simulation

- If the model is simple enough, it may be possible to work with its relationships and quantities to get an exact, mathematical model.

$$-y=mx+c$$

- However, many real systems are very complex. Because of this, their mathematical models also become very complex, and it is not possible to find an exact analytical solution.
- In such cases, the model is studied using simulation. This means the model is tested numerically by giving it different inputs and observing how these inputs affect the system's performance.

- We now classify simulation models along three different dimensions:
- **Static vs. Dynamic Simulation Models:**
 - A static simulation model is a representation of a system at a particular time or a system in which time does not play any role. It does not consider how the system changes with time.
 - On the other hand, a dynamic simulation model represents a system as it evolves over time. It shows how the system state changes with time, such as the movement of items on a conveyor system in a factory.

- **Deterministic vs. Stochastic Simulation Models:**

- A simulation model is called deterministic if it does not include any probabilistic (random) components. In deterministic models, the output is “determined” once the set of input quantities and relationships in the model have been specified.

- In contrast, many real systems involve some degree of randomness in their inputs. These systems are represented using stochastic simulation models. Most queueing systems are modeled stochastically. Since stochastic models include random variables, their outputs are also random and cannot be exact. Therefore, the results obtained from stochastic simulation models are treated only as estimates of the true behavior of the system

- **Continuous vs. Discrete Simulation Models:**
 - A continuous simulation model represents a system in which the state variables change continuously over time. The changes happen smoothly, not at specific moments.
 - A discrete simulation model, on the other hand, represents a system in which changes in the system state occur only at specific points in time, usually when certain events happen, such as arrivals or departures in a queueing system.
 - It should be mentioned that a discrete model is not always used to model a discrete system, and a continuous model is not always used to model a continuous system.

DISCRETE-EVENT SIMULATION

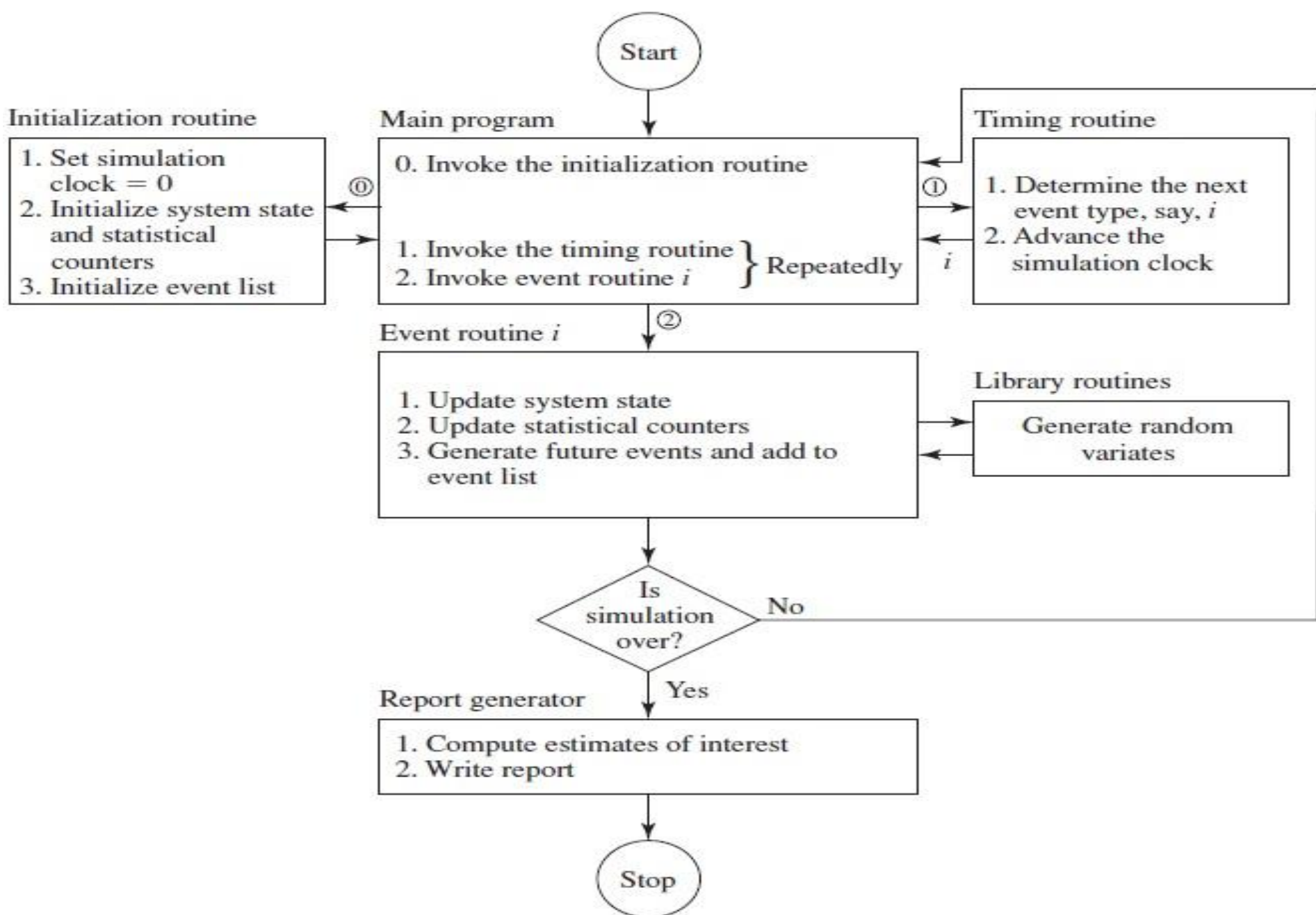
- Discrete- event simulation concerns the modeling a system as it changes over time. In this approach, the system' s state variables do not change continuously; instead, they change instantly at specific points in time when certain events occur.
- Although discrete- event simulation can theoretically be done using hand calculations, in real- world systems the amount of data that needs to be stored and processed is very large. Because of this, discrete- event simulations are usually carried out on a digital computer

- Consider a service facility with a single server—e.g., a one-operator barbershop or an information desk at an airport—for which we would like to estimate the (expected) average delay in queue (line) of arriving customers, where the delay in queue is the length of the time interval from his arrival at the facility to he begins being served.
- For the objective of estimating the average delay of a customer, the state variables for a discrete-event simulation model of the facility would be the status of the server, i.e., either idle or busy, the number of customers waiting in queue to be served (if any), and the time of arrival of each person waiting in queue.

Components and Organization of a Discrete-Event Simulation Model

- **System state:** The collection of state variables necessary to describe the system at a particular time
- **Simulation clock:** A variable giving the current value of simulated time
- **Event list:** A list containing the next time when each type of event will occur
- **Statistical counters:** Variables used for storing statistical information about system performance
- **Initialization routine:** A subprogram to initialize the simulation model at time 0

- **Timing routine:** A subprogram that determines the next event from the event list and then advances the simulation clock to the time when that event is to occur
- **Event routine:** A subprogram that updates the system state when a particular type of event occurs
- **Library routines:** A set of subprograms used to generate random observations from probability distributions that were determined as part of the simulation model



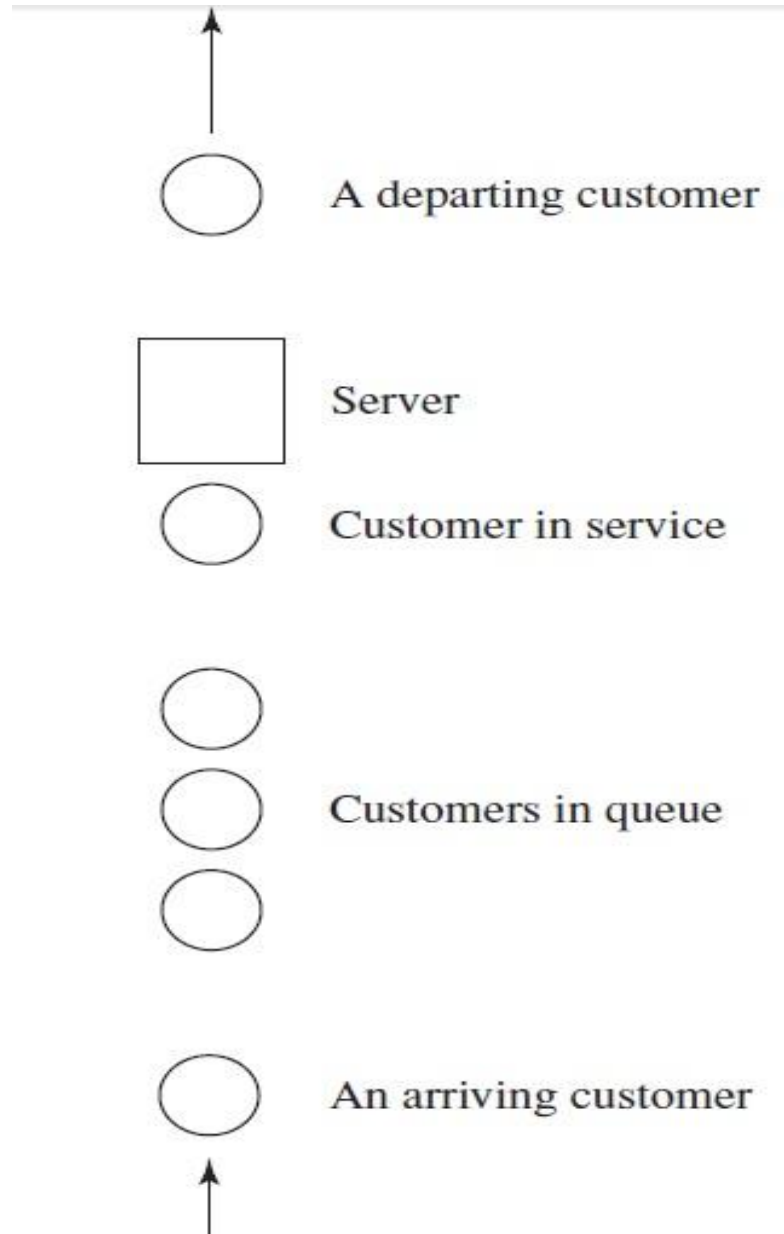
- A system is a well-defined collection of entities. Entities are characterized by data values called attributes, and these attributes are part of the system state for a discrete-event simulation model.
- The server has the attribute “server status” (busy or idle), and the customers waiting in queue have the attribute “time of arrival.” (The number of customers in the queue might also be considered an attribute of the server.)

SIMULATION OF A SINGLE-SERVER QUEUEING SYSTEM

- We now show in detail how to simulate a single-server queueing system such as a one-operator barbershop.
- Although this system looks very simple compared to real-world systems, the way it is simulated clearly shows how even very complex simulation systems actually work.

Problem Statement

- Consider a single-server queueing system (see the next Figure) for which the interarrival times $A_1, A_2, \dots$ are independent and identically distributed (IID) random variables.
- “Identically distributed” means that the interarrival times have the same probability distribution.
- A customer who arrives and finds the server idle enters service immediately, and the service times $S_1, S_2, \dots$ of the successive customers are IID random variables that are independent of the interarrival times.



- A customer who arrives and finds the server busy joins the end of a single queue. When the server finishes service for a customer, it selects the next customer from the queue (if any) using the first-in, first-out (FIFO) rule.
- The simulation will start in the empty- and-idle state; that is, there are no customers in the system and the server is idle.
- At time 0, we will start waiting for the arrival of the first customer, which will happen after the first interarrival time, A_1 , instead of at time 0 (this could also be a valid but different modeling assumption).

- We want to simulate this system until a fixed number (n) of customers have completed their delays in queue.
- Note that the time at which the simulation ends is a random variable, because it depends on the observed values of the interarrival and service-time random variables.

- To measure the performance of this system, we will look at estimates of three quantities. First, we will estimate the expected average delay in queue of the n customers **completing their delays** during the simulation; we denote this quantity by $d(n)$.
- The word “expected” in the definition of $d(n)$ means this: in one run of the simulation, the average delay observed for the n customers depends on the interarrival and service-time random variables.

- On another run of the simulation (or on a different day for the real system) there would probably be arrivals at different times, and the service times required would also be different; this would give a different value for the average of the n delays.
- So, the average delay from one simulation run is not fixed; it can change each time the simulation is run. Because of this, the average delay is treated as a random variable.
- From a single run of the simulation resulting in customer delays $D_1, D_2, \dots, D_n$, an obvious estimator of $d(n)$ is:

$$\hat{d}(n) = \frac{\sum_{i=1}^n D_i}{n}$$

- which is just the average of the n D_i 's that were observed in the simulation.

- While an estimate of $d(n)$ provides information about system performance from the customers' point of view, the management of such a system may need different information.
- Because most real simulations are quite complex and can be time-consuming to run, we usually collect many output measures of performance, which describe different aspects of system behavior.

- One such measure for our simple model is the expected average number of customers in the queue (but not being served), which is denoted by $q(n)$...

- This is a different type of “average” compared to the average delay in queue, because it is taken over (continuous) time rather than over customers (which are discrete).
- Therefore, we need to clearly define what is meant by the time-average number of customers in queue.
- To do this, let $Q(t)$ denote the number of customers in queue at time t , for any real number $t \geq 0$, and let $T(n)$ be the time required to observe our n delays in queue.

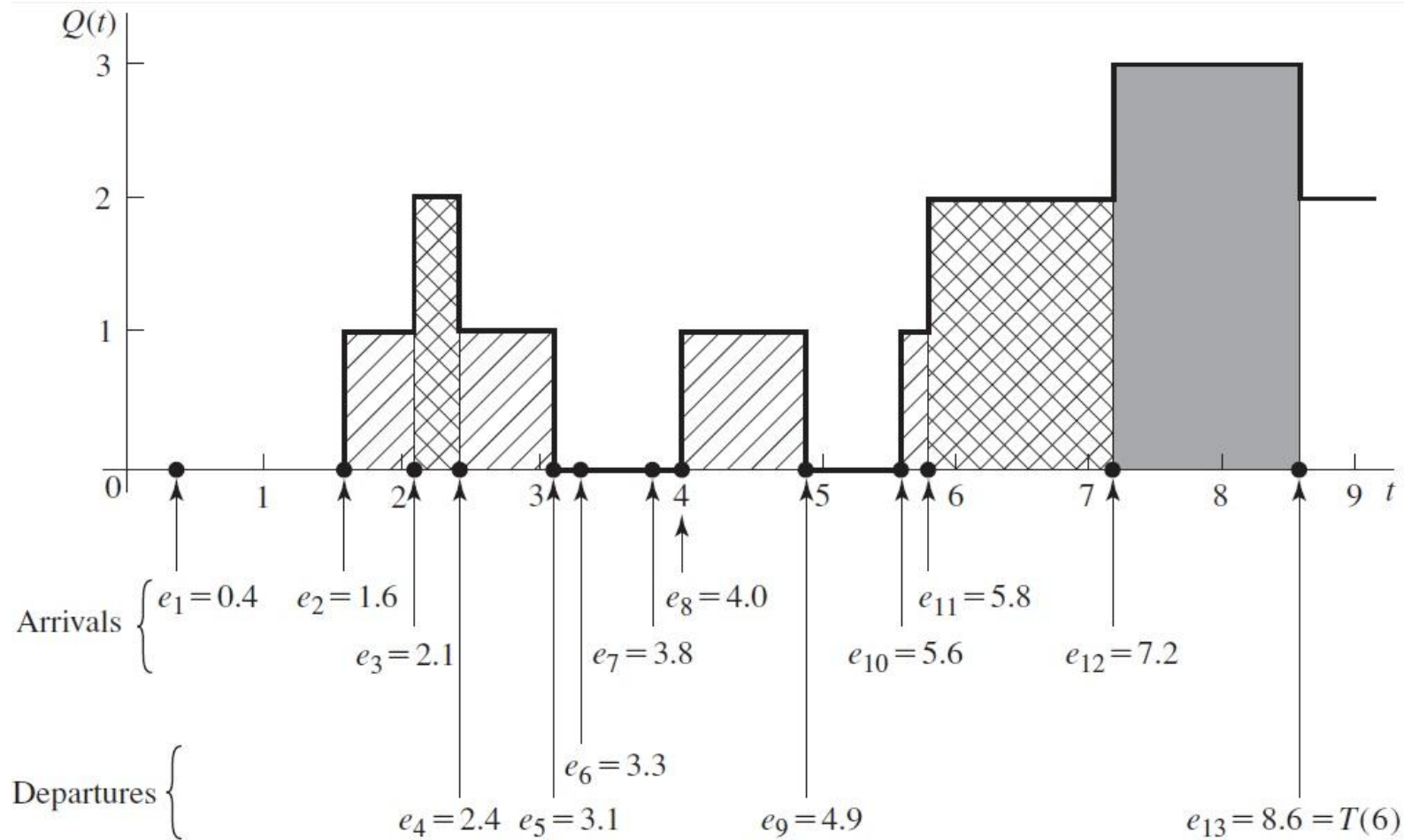
- Then for any time t between 0 and $T(n)$, $Q(t)$ is a nonnegative integer. Further, if we let p be the expected proportion (which will be between 0 and 1) of the time that $Q(t)$ is equal to i , then a reasonable definition of $q(n)$ would be:

$$q(n) = \sum_{i=0}^{\infty} ip_i$$

- If we let T_i be the total time during the simulation when the queue length is i , then $T(n) = T_0 + T_1 + T_2 + \dots$ and $p_i = T_i/T(n)$ so that we can rewrite the above equation as:

$$\hat{q}(n) = \frac{\sum_{i=0}^{\infty} iT_i}{T(n)}$$

- The next figure illustrates a possible time path, or realization, of $Q(t)$ for this system in the case of $n = 6$.
- Arrivals occur at times 0.4, 1.6, 2.1, 3.8, 4.0, 5.6, 5.8, and 7.2. Departures (service completions) occur at times 2.4, 3.1, 3.3, 4.9, and 8.6, and the simulation ends at time $T(6) = 8.6$.



- Remember in looking at the previous figure that $Q(t)$ does not count the customer in service (if any), so between times 0.4 and 1.6 there is one customer in the system being served, even though the queue is empty [$Q(t) = 0$].
- The same is true between times 3.1 and 3.3, between times 3.8 and 4.0, and between times 4.9 and 5.6.
- Between times 3.3 and 3.8, however, the system is empty of customers and the server is idle, as is obviously the case between times 0 and 0.4.
- To compute $q(n)$, we must first compute the T_i 's over which $Q(t)$ is equal to 0, 1, 2, and so on:

$$T_0 = (1.6 - 0.0) + (4.0 - 3.1) + (5.6 - 4.9) = 3.2$$

$$T_1 = (2.1 - 1.6) + (3.1 - 2.4) + (4.9 - 4.0) + (5.8 - 5.6) = 2.3$$

$$T_2 = (2.4 - 2.1) + (7.2 - 5.8) = 1.7$$

$$T_3 = (8.6 - 7.2) = 1.4$$

($T_i = 0$ for $i \geq 4$, since the queue never grew to those lengths in this realization.)
The numerator in Eq. (1.2) is thus

$$\sum_{i=0}^{\infty} iT_i = (0 \times 3.2) + (1 \times 2.3) + (2 \times 1.7) + (3 \times 1.4) = 9.9 \quad (1.3)$$

- and so our estimate of the time-average number in queue from this particular simulation run is $\hat{q}(6) = 9.9/8.6 = 1.15$

SIMULATION OF AN INVENTORY SYSTEM

- We shall now see how simulation can be used to compare alternative ordering policies for an inventory system.
- Problem Statement:
 - A company sells one important product and wants to decide how many items to keep in stock for each of the next n months, where n is a fixed number
 - The time gap between customer demands is random. These times are independent and identically distributed (IID) and follow an exponential distribution, with an average of 0.1 month between demands.

SIMULATION OF AN INVENTORY SYSTEM

- Problem Statement:

– The amount of demand (D) is random and does not depend on when the demand happens. Each time a demand occurs, it can be

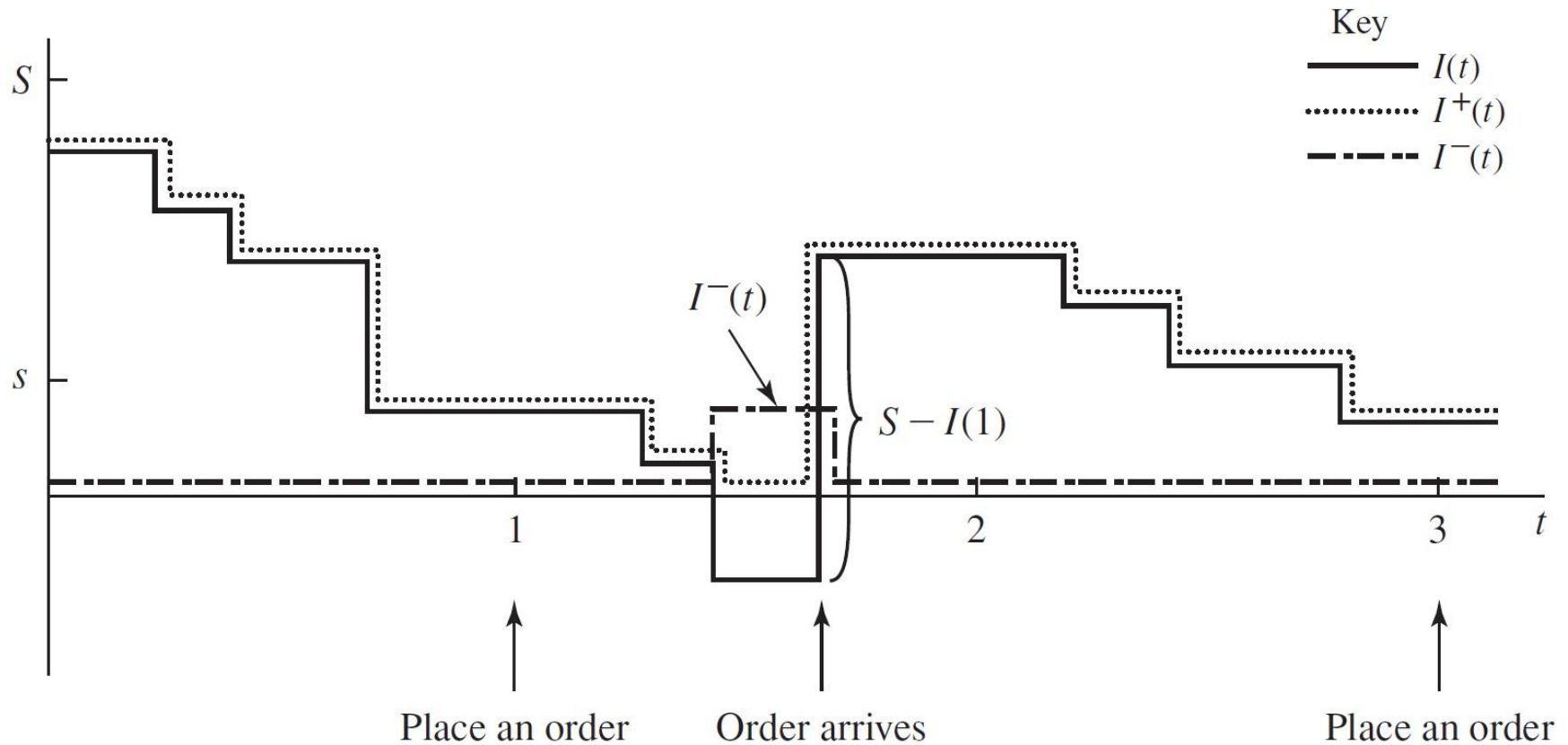
$$D = \begin{cases} 1 & \text{w.p. } \frac{1}{6} \\ 2 & \text{w.p. } \frac{1}{3} \\ 3 & \text{w.p. } \frac{1}{3} \\ 4 & \text{w.p. } \frac{1}{6} \end{cases}$$

So, every demand can ask for a different number of items, and these quantities follow the given probabilities.

SIMULATION OF AN INVENTORY SYSTEM

- Problem Statement:
 - At the start of each month, the company checks how many items are left in inventory and then decides how many items to order from the supplier.
 - If the company orders Z items, it must pay a total cost of $K + iZ$, where $K = \$32$ is the setup cost and $i = \$3$ is the incremental cost per item ordered. If $Z = 0$, meaning no order is placed, then no cost is paid.
 - When an order is placed, the time required for it to arrive (called the delivery lag or lead time) is a random variable that is distributed uniformly between 0.5 and 1 month.

SIMULATION OF AN INVENTORY SYSTEM



SIMULATION OF AN INVENTORY SYSTEM

- The company uses a stationary (s, S) policy to decide how much to order, i.e.,

$$Z = \begin{cases} S - I & \text{if } I < s \\ 0 & \text{if } I \geq s \end{cases}$$

- where I is the inventory level at the beginning of the month

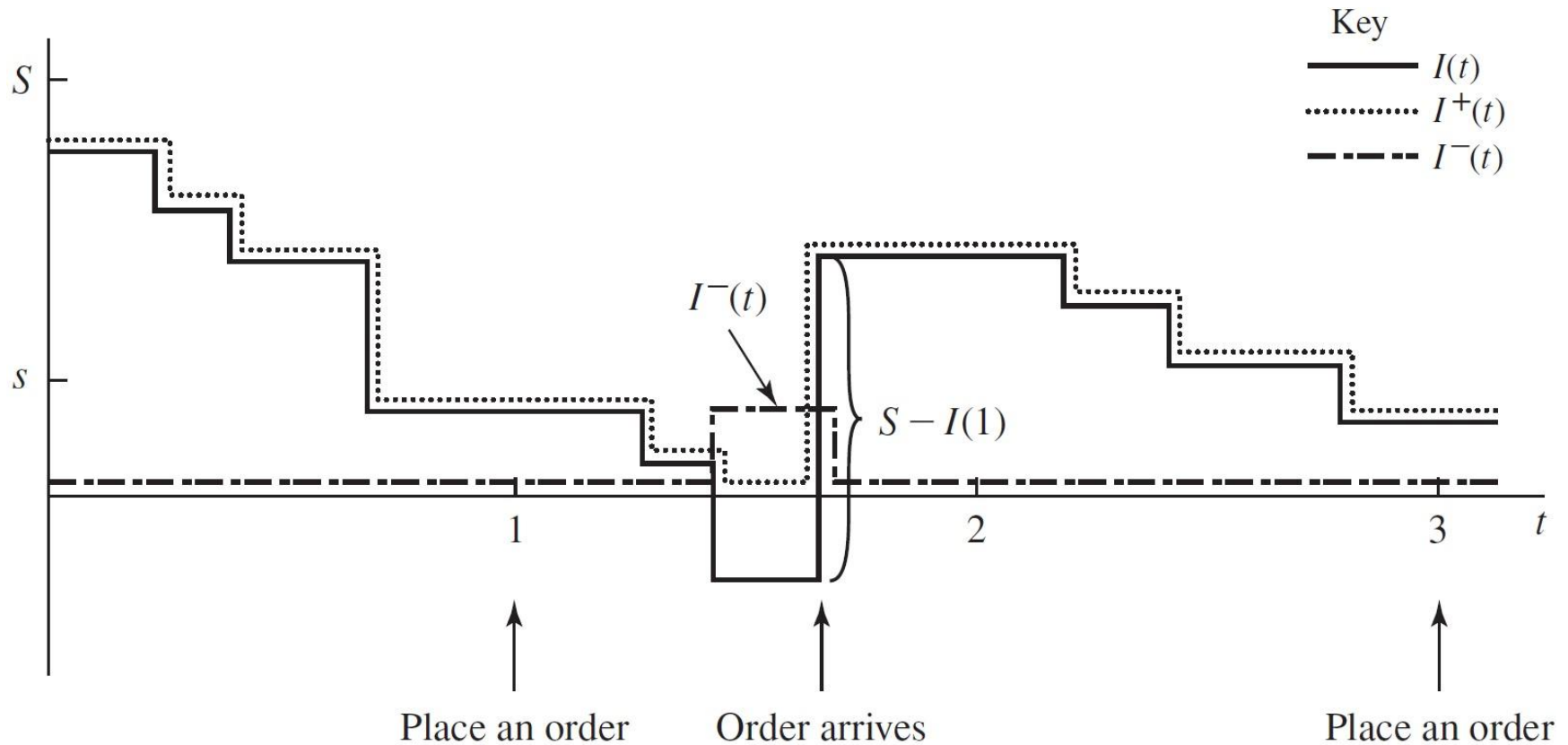
SIMULATION OF AN INVENTORY SYSTEM

- When a demand occurs, it is satisfied immediately if the inventory level is at least as large as the demand.
- If the demand is greater than the inventory, the extra unmet demand is backlogged and will be satisfied later when new items arrive. In this case, the new inventory level is equal to the old inventory level minus the demand size, resulting in a negative inventory level.
- When a new order arrives, the items are first used to clear the backlog (if there is any). Any remaining items are then added to the inventory.

SIMULATION OF AN INVENTORY SYSTEM

- So far, we have discussed only one type of cost incurred by the inventory system, the ordering cost.
- However, most real inventory systems also have two additional types of costs, holding and shortage costs, which we discuss after introducing some additional notation.
- Let $I(t)$ represent the inventory level at time t . The value of $I(t)$ can be positive, zero, or negative.
- Let $I^+(t) = \max\{I(t), 0\}$ be the number of items physically on hand in the inventory at time t [note that $I^+(t) \geq 0$]; and let $I^-(t) = \max\{-I(t), 0\}$ be the backlog at time t [$I^-(t) \geq 0$ as well].
- A possible realization of $I(t)$, $I^+(t)$, and $I^-(t)$ is shown in the next figure. The time points at which $I(t)$ decreases are the ones at which demands occur.

SIMULATION OF AN INVENTORY SYSTEM



SIMULATION OF AN INVENTORY SYSTEM

- In this model, we assume that the company pays a holding cost of $h=\$1$ per item per month for every item kept in positive inventory.
- The holding cost includes expenses such as warehouse rent, insurance, taxes, maintenance, and also the opportunity cost of money being tied up in inventory instead of being invested elsewhere.
- We ignore the fact that some holding costs may still exist even when $I^+(t) = 0$ (that is, when no items are in stock).
- However, since our main goal is to compare different ordering policies, ignoring this cost does not affect the comparison, because this cost is independent of the policy used.

SIMULATION OF AN INVENTORY SYSTEM

- Now, since $I^+(t)$ is the number of items held in inventory at time t , the time-average (per month) number of items held in inventory for the n -month period is:

$$\bar{I}^+ = \frac{\int_0^n I^+(t) dt}{n}$$

- This definition is similar to the earlier definition of the time-average number of customers in a queue. Therefore, the average holding cost per month is equal to $h\bar{I}^+$. where $\bar{I}^+$ is the time-average number of items kept in inventory.

SIMULATION OF AN INVENTORY SYSTEM

- Similarly, suppose the company pays a backlog cost of $\pi = \$5$ per item per month for items that are backlogged. This cost covers extra record keeping and the loss of customers' goodwill when demands are not met on time. The time- average number of items in backlog is denoted by:

$$\bar{I}^- = \frac{\int_0^n I^-(t) dt}{n}$$

- so the average backlog cost per month is $\pi \bar{I}^-$.

SIMULATION OF AN INVENTORY SYSTEM

- Assume that the initial inventory level is $I(0)=60$ and that no order is outstanding.
- We simulate the inventory system for $n=120$ months and use the average total cost per month (which is the sum of the average ordering cost per month, the average holding cost per month, and the average shortage cost per month) to compare the following nine inventory policies:

s	20	20	20	20	40	40	40	60	60
S	40	60	80	100	60	80	100	80	100

Program Organization and Logic

- Our model of the inventory system uses the following types of events:

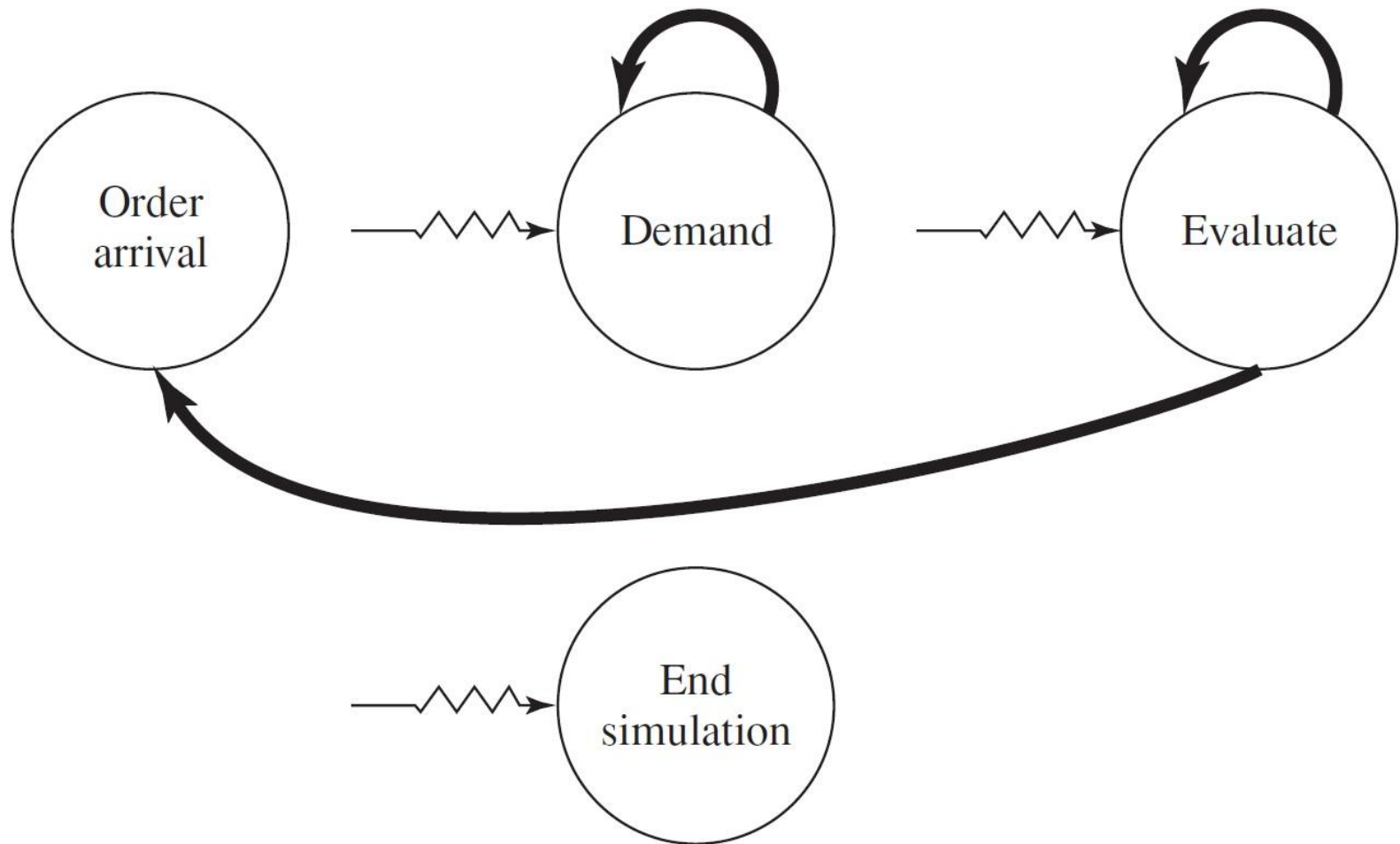
Event description	Event type
Arrival of an order to the company from the supplier	1
Demand for the product from a customer	2
End of the simulation after n months	3
Inventory evaluation (and possible ordering) at the beginning of a month	4

- We have chosen to make the end of the simulation event type 3 rather than type 4, since at time 120 both “end-simulation” and “inventory-evaluation” events will eventually be scheduled and we would like to execute the former event first at this time (Since the simulation is over at time 120, there is no sense in evaluating the inventory and possibly ordering, incurring an ordering cost for an order that will never arrive).

Program Organization and Logic

- The execution of event type 3 before event type 4 is guaranteed because the timing routine gives preference to the lowest-numbered event if two or more events are scheduled to occur at the same time.
- In general, a simulation model should be designed to process events in an appropriate order when time ties occur.
- An event graph appears in the next Figure.

Program Organization and Logic



Program Organization and Logic

- There are three types of random variates needed to simulate this system. The interdemand times are distributed exponentially.
- The demand-size random variate D must be discrete, as described above, and can be generated as follows.
- First divide the unit interval into the contiguous subintervals $C1 = [0, 1/6]$, $C2 = [1/6, 1/2]$, $C3 = [1/2, 5/6]$ and $C4 = [5/6, 1]$, and obtain a $U(0, 1)$ random variate U from the random-number generator.

Program Organization and Logic

- If U falls in C_1 , return $D = 1$; if U falls in C_2 , return $D = 2$; and so on.
- Since the width of C_1 is $1/6 - 0 = 1/6$, and since U is uniformly distributed over $[0, 1]$, the probability that U falls in C_1 (and thus that we return $D = 1$) is $1/6$; this agrees with the desired probability that $D = 1$.
- Similarly, we return $D = 2$ if U falls in C_2 , having probability equal to the width of C_2 , $1/2 - 1/6 = 1/3$, as desired; and so on for the other intervals.
- The subprogram to generate the demand sizes uses this principle and takes as input the cutoff points defining the above subintervals, which are the cumulative probabilities of the distribution of D .

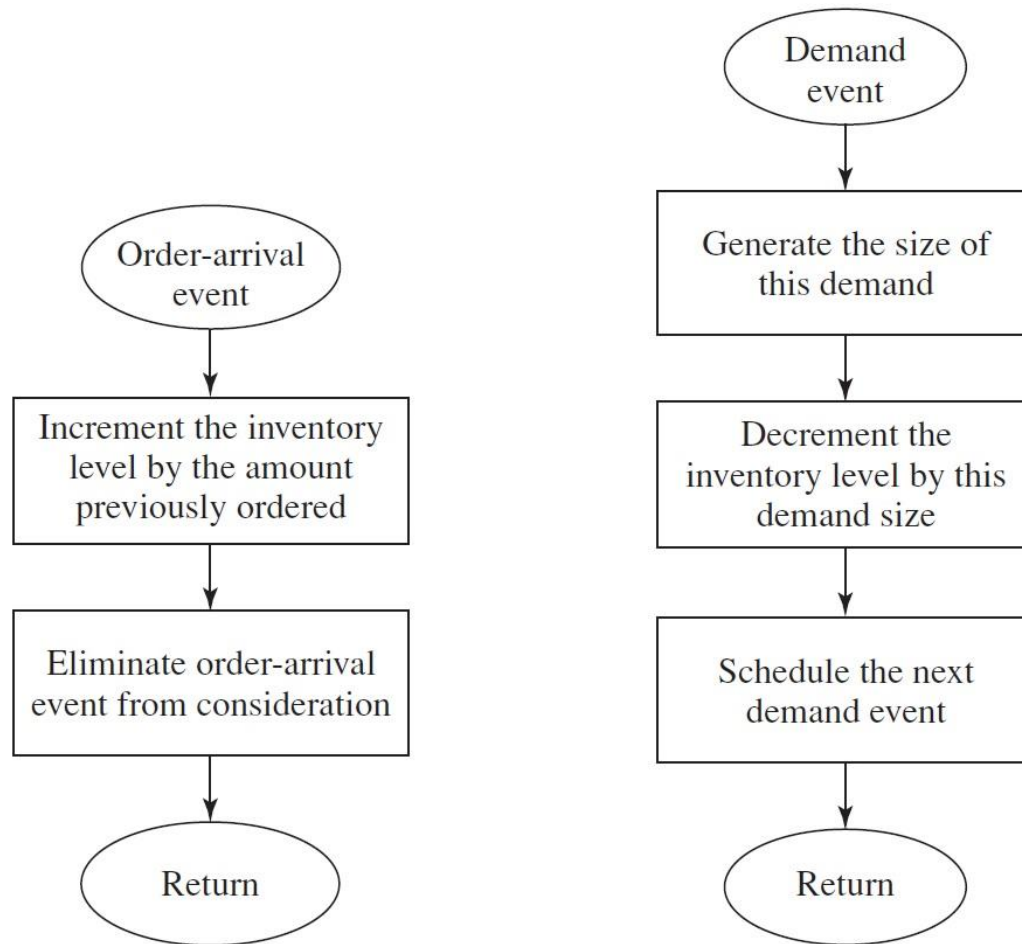
Program Organization and Logic

- We now describe the logic for event types 1, 2, and 4, which actually involve state changes.
- The order-arrival event is flowcharted in the next figure, and must make the changes necessary when an order (which was previously placed) arrives from the supplier.
- The inventory level is increased by the amount of the order, and the order-arrival event must be eliminated from consideration.
- A flowchart for the demand event is also given in the next figure, and processes the changes necessary to represent a demand's occurrence.

Program Organization and Logic

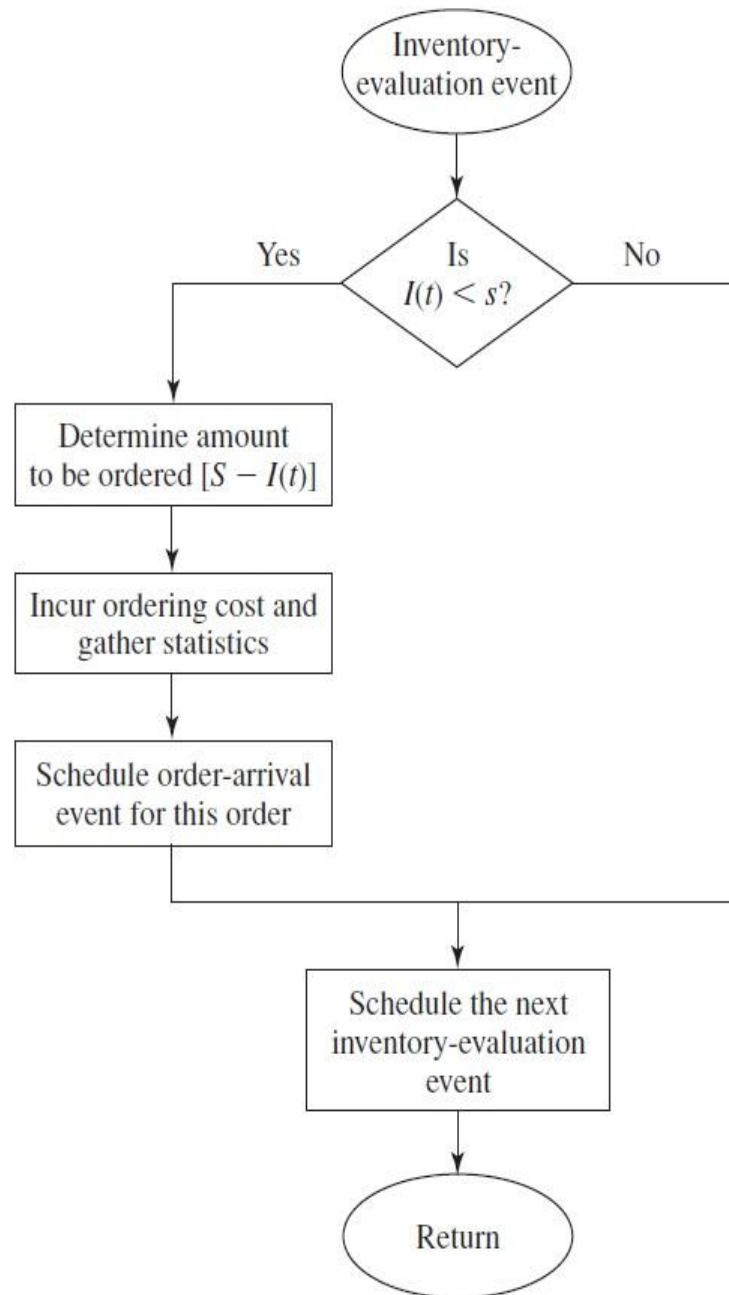
- First, the demand size is generated, and the inventory is decremented by this amount.
- Finally, the time of the next demand is scheduled into the event list. Note that this is the place where the inventory level might become negative.

Program Organization and Logic



Program Organization and Logic

- The inventory-evaluation event, which takes place at the beginning of each month, is flowcharted in the next figure.
- If the inventory level $I(t)$ at the time of the evaluation is at least s , then no order is placed, and nothing is done except to schedule the next evaluation into the event list.
- On the other hand, if $I(t) < s$, we want to place an order for $S - I(t)$ items.
- This is done by storing the amount of the order $[S - I(t)]$ until the order arrives, and scheduling its arrival time.
- In this case as well, we want to schedule the next inventory-evaluation event.



Program Organization and Logic

- It is convenient to write a separate nonevent routine to update the continuous-time statistical accumulators which is flowcharted in the next figure.
- The principal issue is whether we need to update the area under $I^-(t)$ or $I^+(t)$ (or neither).
- If the inventory level since the last event has been negative, then we have been in backlog, so the area under $I^-(t)$ only should be updated. On the other hand, if the inventory level has been positive, we need only update the area under $I^+(t)$.
- If the inventory level has been zero (a possibility), then neither update is needed.
- The code for this routine also brings the variable for the time of the last event up to the present time. This routine will be invoked from the main program just after returning from the timing routine, regardless of the event type or whether the inventory level is actually changing at this point.
- This provides a simple (if not the most computationally efficient) way of updating integrals for continuous-time statistics.

